

DEEPPFAKE IMAGE DETECTION

PROJECT REPORT

Submitted by:

Anushka

Student & Student AI Research Engineer

Email: anuushka27@gmail.com

GitHub: <https://github.com/anuushka27-maker/Deepfake-Image-Detection>

ABSTRACT

Deepfake technology allows artificial intelligence to create fake images of people that look extremely real. These images are generated using deep learning models, and they can be difficult to identify with the human eye. While deepfakes can be useful in creative fields such as movies and education, they also have a negative side. They are often used for spreading false information, damaging someone's reputation, online scams, and identity misuse. Because deepfakes are becoming more realistic every year, it is important to develop tools that can help detect them automatically.

In this project, I have worked on building a deepfake image detection system using a deep learning model. The model analyses face images and predicts whether the image is real or fake. The system was trained on a dataset containing both real and manipulated images. The process includes data preprocessing, training the model, testing it, and then evaluating the results using accuracy and performance measures.

The purpose of this project is to understand how deep learning can be applied to detect fake images and to learn the challenges involved in building such systems. This is a student-level research project created to explore the topic and gain experience in computer vision and machine learning. The project is not intended to be a fully commercial solution but serves as a foundation for future improvements and advanced research.

TABLE OF CONTENTS

CHAPTER 1 : INTRODUCTION	5
1.1 Background	
1.2 Industry Relevance and Real-World Need	
1.3 Real Examples & Impact	
1.4 Problem Definition	
1.5 Importance of This Project	
1.6 Motivation	
1.7 Project Objectives	
1.8 Project Scope and Limitations	
1.9 Report Organization	9
CHAPTER 2 : LITERATURE REVIEW	10
2.1 Literature Review	
2.2 Evolution of Deepfake Technology	
2.3 Deepfake Creation Techniques	
2.4 Traditional Deepfake Detection Approaches	
2.5 Deep Learning-Based Detection Approaches	
2.6 Existing Research Trends & Industry Work	
2.7 Challenges in Deepfake Detection	
2.8 Deepfake Detection in Cybersecurity & Law Enforcement	
2.9 Need for Hybrid Detection Approaches	
2.10 Summary	15
CHAPTER 3 : METHODOLOGY / PROPOSED SYSTEM	16
3.1 System Overview	
3.2 Problem Environment	
3.3 System Architecture	
3.4 Hybrid Model Architecture (CNN + ResNet50)	
3.5 Dataset Pipeline and Preprocessing	
3.6 Training Strategy and Optimization	
3.7 Callbacks and Monitoring	
3.8 Implementation Environment	
3.9 Evaluation Metrics	
3.10 Deployment Using Streamlit	
3.11 Limitations and Real-World Considerations	
3.12 Summary	23
CHAPTER 4 : DATASET & IMPLEMENTATION	24
4.1 Introduction	
4.2 Dataset Description	
4.3 Dataset Organization and Directory Structure	
4.4 Data Preprocessing Pipeline	
4.5 Implementation Pipeline	
4.6 Challenges Faced	
4.7 Summary	29

CHAPTER 5 : RESULTS AND EVALUATION	30
5.1 Introduction	
5.2 Accuracy and Loss Analysis	
5.3 Confusion Matrix Evaluation	
5.4 ROC and Precision–Recall Curve	
5.5 Sample Prediction Outputs	
5.6 Comparative Evaluation	
5.7 Discussion	
5.8 Summary	34
CHAPTER 6 : CONCLUSION & FUTURE WORK	35
6.1 Conclusion	
6.2 Key Outcomes	
6.3 Future Scope	
6.4 Closing Remarks	38
APPENDIX	44

CHAPTER-1

INTRODUCTION

1.1 BACKGROUND

The rapid growth of artificial intelligence has transformed how digital content is created, shared, and consumed. Deep learning techniques, especially generative models, have made it possible to produce synthetic images that appear visually real. One such development is **deepfake technology**, which enables the creation of manipulated images that replicate a person's face with a high degree of realism. As a result, distinguishing real content from fake has become increasingly difficult.

Deepfakes were initially developed for creative purposes such as film production, digital special effects, animations, entertainment, and accessibility tools. However, as the technology advanced and became more accessible, deepfakes started to be used for harmful activities. Today, deepfakes are involved in cybercrime, misinformation campaigns, financial fraud, political manipulation, and online harassment. With the increasing spread of digitally altered images on social media platforms, identity security and privacy protection have become major concerns.

As fake digital media becomes more common and convincing, the need for reliable detection systems is stronger than ever. Manual inspection is no longer effective, especially when thousands of images circulate online every second. Automated deepfake detection solutions are now essential for cybersecurity, law enforcement investigations, digital forensics, and platform safety systems.

1.2 Industry Relevance and Real-World Need

Deepfake detection is not just an academic topic — it is a real industry challenge. Organizations and sectors across the world are actively searching for reliable systems that can

verify image authenticity before it is used or published. Some practical areas where deepfake detection plays a crucial role include:

- **Law enforcement & digital forensics:** Authenticating visual evidence and crime data
- **Cybersecurity and digital identity protection:** Preventing impersonation and fraud
- **Social media and content moderation systems:** Detecting manipulated images before user posting
- **News and journalism:** Verifying visual content to avoid misleading information
- **Corporate security:** Preventing phishing, CEO impersonation, and business scams

Global incidents involving fake media are increasing, and several legal cases have already proven the damage that deepfakes can cause. Companies like Meta, Google, Adobe, and Microsoft are investing heavily in detecting manipulated media. This proves that deepfake detection is a necessary and growing field with a strong demand for skilled engineers.

1.3 Real Examples & Impact

A number of real-world incidents have shown how dangerous deepfakes can be:

- Scammers impersonated business executives using deepfake images and voice to steal money from companies.
- Misleading political deepfakes were used to influence public opinion during elections.
- Individuals have been targeted using manipulated face images to damage reputation and cause emotional or legal trouble.
- Fake celebrity and public figure content is shared widely on social platforms, leading to exploitation and harassment.

These examples reflect why deepfake detection systems need to exist and why this problem cannot be ignored. The goal is not only technological innovation, but also public safety and trust in digital media.

1.4 Problem Definition

The core problem addressed in this project is:

To build an automated system capable of identifying whether an input face image is real or deepfake, with reliable accuracy and performance suitable for real usage and deployment on a personal system.

Traditional visual inspection or simple image processing methods no longer work because deepfake images are almost indistinguishable to the human eye. Therefore, an AI-driven approach is required to automatically analyze image patterns, textures, and inconsistencies that humans cannot detect.

1.5 Importance of This Project

This project is important because it contributes to digital security and helps protect society from the negative consequences of manipulated media. By creating a working prototype that can detect deepfake images, this project supports:

- Individuals who wish to protect their identity and privacy
- Organizations that need to verify digital content
- Law enforcement teams requiring forensic image authentication
- Social platforms that need automated content moderation tools

Even though this system is developed at a student level, it represents a practical step toward real-world cybersecurity solutions.

1.6 Motivation

The motivation for choosing this project comes from both personal interest in artificial intelligence and the real-world urgency of deepfake detection. Working independently on this project provides hands-on experience in:

- Building practical AI applications
- Understanding deep learning model behavior
- Handling real datasets and classification tasks
- Preparing a project prototype suitable for deployment on a local machine
- Gaining portfolio experience for internships and future opportunities

The project is designed to not only learn theory but to build something functional and valuable.

1.7 Project Objectives

The key objectives of this project are:

- To understand how deepfake images are created and why they are difficult to detect
- To develop a deep learning-based model that can classify images as real or fake
- To train and evaluate the system using a labeled dataset
- To create a near real-deployment prototype that runs on a personal machine
- To analyze performance using accuracy and measurement graphs
- To document the full development lifecycle in an industry-standard project report

1.8 Project Scope and Limitations

This system currently focuses only on **deepfake image detection**. It does not include advanced real-time video analysis or audio deepfake detection. The performance of the model depends on dataset size and available hardware. Since the project is developed by a single student researcher, it has certain resource limitations, but the prototype is designed to be extendable and upgradable in the future.

This project is intended as a practical engineering solution rather than a fully commercial or enterprise-level tool. It demonstrates the feasibility of deploying deepfake detection locally on a computer without cloud dependency.

1.9 Report Organization

This document is structured as follows:

- **Chapter 1** introduces the project background, relevance, motivation, and objectives.
- **Chapter 2** presents a detailed literature review and existing deepfake detection approaches.

- **Chapter 3** explains the proposed system architecture and methodology.
- **Chapter 4** describes dataset handling, implementation steps, and training workflow.
- **Chapter 5** presents experimental results and performance evaluation.
- **Chapter 6** concludes the work and discusses future improvements.

CHAPTER-2

LITERATURE REVIEW

2.1 Literature Review

Deepfake technology has become one of the most influential and concerning developments within artificial intelligence. This technology uses deep learning models to manipulate or generate highly realistic human faces that appear real, making it extremely challenging to identify authenticity using traditional inspection techniques. As deepfake creation tools improve in quality, deepfake detection has become a critical requirement for cybersecurity, digital forensics, law enforcement, content moderation systems, and responsible AI projects.

This chapter reviews the technological background of deepfake creation, the evolution of detection methods, the current research landscape, real-world industrial solutions, and the challenges faced in accurately detecting manipulated images.

2.2 Evolution of Deepfake Technology

The earliest deepfake implementations were based on **autoencoders**, which learned facial feature representation and reconstructed faces on different identities. The breakthrough came with the introduction of **Generative Adversarial Networks (GANs)**, where two neural networks compete against each other—one generating synthetic images and the other distinguishing real from fake. This adversarial training mechanism significantly improved visual authenticity.

Over time, deepfake models evolved into:

- **StyleGAN, StyleGAN2 & StyleGAN3**, capable of generating ultra-realistic synthetic human faces from scratch

- **CycleGAN** for domain translation without paired datasets
- **Diffusion Models** used for generative content creation in tools such as Stable Diffusion and Midjourney
- **Transformer-based Vision models** which improved image consistency and realism

As a result, deepfakes have become more convincing, requiring more advanced detection approaches.

2.3 Deepfake Creation Techniques

Some widely known deepfake creation strategies include:

Face Swapping

Replacing one person's face with another in an image or video while maintaining original expressions and positioning.

Face Reenactment

Copying expressions, emotions, and movements from a source person to a target face.

Synthetic Face Generation

Creating completely artificial faces that do not belong to any real person, popularly used in AI-generated profiles.

Face Blending & Morphing

Combining facial characteristics of two identities into a single synthetic output.

These techniques are advancing continuously and are now being used at scale in consumer-level applications.

2.4 Traditional Deepfake Detection Approaches

Early detection research relied heavily on visual inconsistencies caused by imperfect deepfake generation. Traditional methods analyzed:

- Odd blinking frequency or eye movement artifacting
- Poorly aligned facial boundaries
- Color mismatch or inconsistent lighting
- Skin texture irregularities
- Head movements and unnatural posture

Although these techniques provided initial success, modern deepfake generators eliminate most visual defects, making traditional detection nearly ineffective.

2.5 Deep Learning-Based Detection Approaches

With increasing deepfake realism, researchers shifted toward automated **deep learning**-based detection frameworks capable of identifying hidden patterns at feature level. Common approaches include:

CATEGORY	EXPLANATION
CNN-based classifiers	Detect texture inconsistencies, blending artifacts, and generation noise
Transfer learning using models such as ResNet, Xception, EfficientNet	Pre-trained feature extraction to improve performance with smaller datasets
MesoNet architectures	Lightweight models for detecting subtle artifacts
Frequency domain analysis	Identifies manipulation in DCT/Fourier transformation space
Transformer and attention-based networks	Capture global relationships beyond local pixel patterns

Ensemble & hybrid models	Combine features from multiple models for improved accuracy
-------------------------------------	---

These approaches outperform traditional methods, especially when deepfakes become visually flawless.

2.6 Existing Research Trends & Industry Work

Many tech companies and research institutions are working on deepfake detection solutions. Examples include:

- **Facebook Deepfake Detection Challenge (DFDC)** encouraged global research for robust detection systems.
- **Google Jigsaw, Microsoft Video Authenticator, and Adobe Content Authenticity Initiative** focus on real-time verification and media authenticity.
- **News & forensic agencies** use automated tools to verify online content.

Despite progress, achieving a universally reliable deepfake detector remains unsolved due to model generalization challenges.

2.7 Challenges in Deepfake Detection

Detecting deepfakes remains difficult due to several critical challenges:

- Rapid advancements in deepfake generation tools
- Domain shift and generalization issues when encountering unseen data
- Lack of standard datasets covering all fake types
- High computational needs for real-time detection
- Compression artifacts in online platforms masking deepfake traces
- Dataset imbalance and overfitting challenges
- High resource cost for continuous retraining

These factors highlight the need for hybrid, scalable, and adaptable detection techniques.

2.8 Deepfake Detection in Cybersecurity & Law Enforcement

In cybersecurity, deepfake detection protects individuals and organizations from identity-based cyberattacks, fraud attempts, and phishing. In law enforcement, deepfake detection plays a role in verifying digital evidence and preventing manipulated proof from influencing legal processes.

Accurate automated detection supports:

- Criminal investigation & digital forensics
- Court evidence validation
- Corporate communication authentication
- Protection against CEO impersonation and scam frauds

This aligns closely with the focus of this project.

2.9 Need for Hybrid Detection Approaches

Researchers and industry solutions increasingly combine:

- Pretrained feature extractors
- Custom CNN heads
- Regularization + dropout strategies
- Fine-tuning techniques

Hybrid architectures improve both feature extraction depth and classification performance, greatly reducing overfitting and improving generalization.

This aligns well with the **Hybrid CNN + ResNet50** approach used in the proposed system described in Chapter 3.

2.10 Summary

Deepfake technology has evolved quickly from basic autoencoder models to complex GANs, diffusion models, and transformer-based generation. With increasing detection difficulty, automated AI-based techniques are necessary to identify manipulations that cannot be detected visually. Although many deep learning architectures are currently used for detection,

no universal solution exists due to continuous improvement of deepfake generation systems and real-world environmental constraints.

This literature review provides the foundation for understanding the need for a robust detection system, which will be discussed in the next chapter through the design and implementation of a hybrid model for deepfake image detection.

CHAPTER-3

METHODOLOGY/PROPOSED SYSTEM

3.1 System Overview

The proposed system is a deep learning–based solution designed for detecting deepfake facial images by classifying them into two categories: **real** and **fake**. The system uses a **hybrid Convolutional Neural Network (CNN)** architecture that integrates a pretrained **ResNet50** backbone with custom dense layers to enhance classification accuracy while reducing overfitting. The system is developed with practical deployment feasibility in mind, allowing the model to be hosted locally through a **Streamlit-based user interface** for real-time testing and demonstrations.

This chapter describes the complete workflow of the proposed deepfake image detection system, including dataset processing, model architecture, training configuration, performance optimization strategies, and deployment workflow.

3.2 Problem Environment

Deepfakes represent a growing cybersecurity and identity threat, widely used in online fraud, impersonation attacks and misinformation. Manual inspection cannot reliably differentiate manipulated images from real ones, and traditional computer vision techniques fail against high-quality deepfake generation such as StyleGAN or FaceSwap.

Therefore, the system must:

- Automatically detect manipulation patterns hidden within texture, artifacts and feature consistency
- Train effectively on large datasets
- Generalize to diverse image sources and image quality variations
- Prevent overfitting and data leakage during training

- Allow deployment on local consumer hardware for accessibility and testing

The proposed solution addresses these needs through an optimized training pipeline and hybrid deep learning architecture.

3.3 System Architecture

The architecture consists of four major components:

Dataset Management and Preprocessing

Hybrid CNN + ResNet50 Classification Model

Training and Fine-Tuning Strategy

Evaluation and Deployment System

SYSTEM ARCHITECTURE DIAGRAM

System Architecture of Deepfake Image Detection System

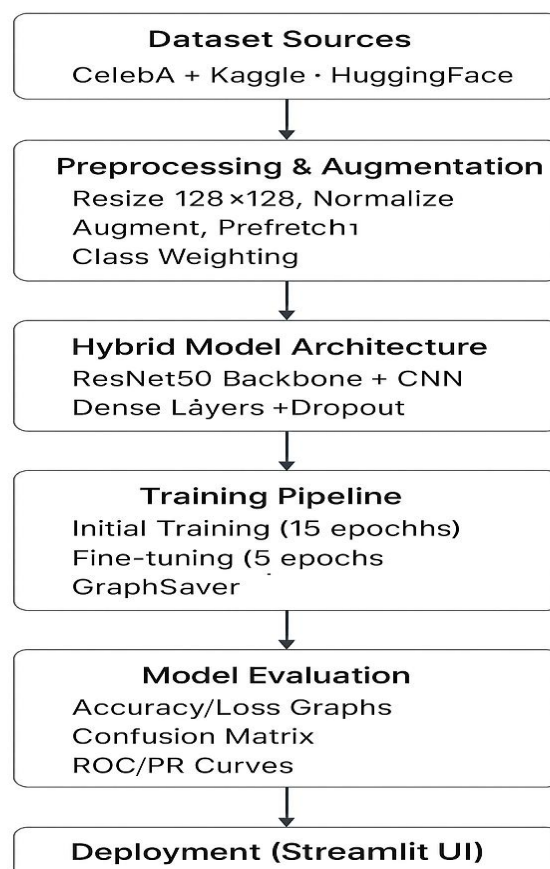


FIG. 3.1 THE SYSTEM ARCHITECTURE

Internal Component Summary

Input images are standardized to dimensions of $128 \times 128 \times 3$

Preprocessing applies normalization and augmentation

ResNet50 backbone extracts high-level semantic features

Custom dense layers perform binary classification

Sigmoid activation outputs probability scores

This modular design ensures clarity, maintainability, and end-to-end traceability across the entire deepfake detection pipeline.

3.4 Hybrid Model Architecture (CNN + ResNet50)

The core classifier uses **TensorFlow 2.15 with Keras** and employs **transfer learning**, leveraging ResNet50 pretrained on the ImageNet dataset. The top layers of ResNet50 are removed and replaced with new dense layers tailored specifically for binary classification.

Transfer learning allows the system to learn high-level visual features without training from scratch.

Key architectural choices:

ResNet50 Base Model Extracts deep visual features efficiently without training from scratch

GlobalAveragePooling2D instead of Flatten to reduce parameters and avoid overfitting

Batch Normalization Improves stability and prevents overfitting

Dropout (0.3 and 0.2) to reduce overfitting and improve generalization

L2 Regularization for better weight control

Sigmoid activation for binary classification

This architecture achieved strong performance while minimizing computational complexity and training time.

BLOCK DIAGRAM OF HYBRID MODEL ARCHITECTURE

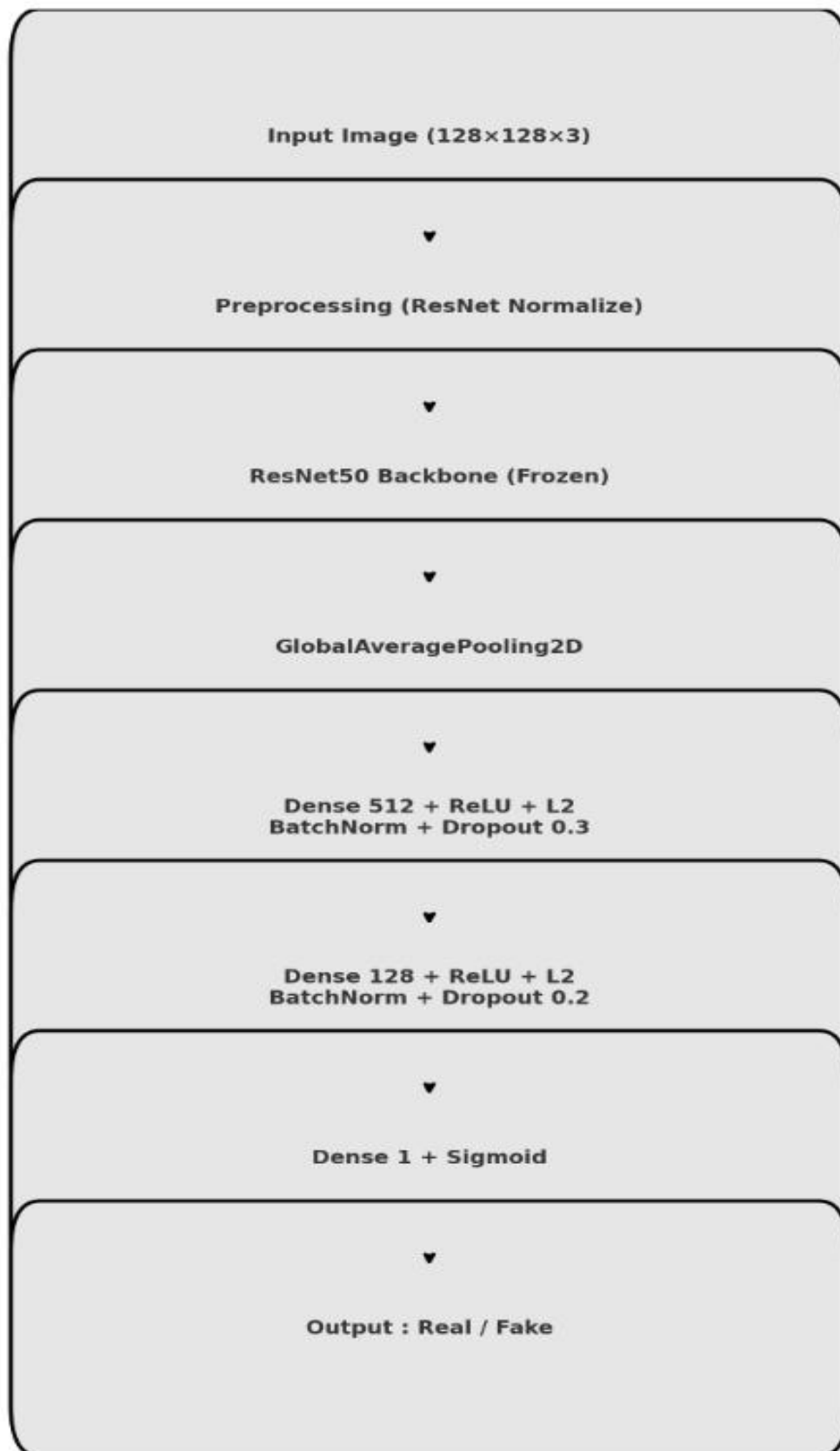


FIG. 3.2 MODEL ARCHITECTURE

3.5 Dataset Pipeline and Preprocessing

The dataset for this project consists of approximately **141,000 images**, collected from multiple reliable public sources including CelebA, Kaggle, and HuggingFace. The dataset includes balanced sets of real and deepfake facial images.

Final Distribution:

TRAINING: 80% (~114,000 IMAGES)

TESTING: 15%(~ 26,000 IMAGES)

VALIDATION: 5%(~6,500 IMAGES)

Preprocessing Procedures

- Resizing images to **128 × 128**
- Normalization using pixel scaling (**img / 255**)
- Dataset augmentation to improve variation:
 - Random horizontal flip, rotation, zoom, brightness & contrast adjustments
- Prefetching using AUTOTUNE to accelerate data loading
- Class weighting to manage dataset imbalance and improve robustness

3.6 Training Strategy and Optimization

The model was trained using a structured two-stage approach to maximize performance:

Phase 1: Initial Training

- ResNet50 layers frozen
- 15 epochs using **Adam optimizer (learning rate = 1e-4, clipnorm=1.0)**
- Loss: Binary Cross-Entropy
- Callbacks: ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
- Graphs saved after every epoch for visual monitoring

Phase 2: Fine-Tuning

- Last **20 layers** of ResNet50 unfrozen
- Low learning rate (**1e-5**) to avoid destructive updates
- **5** additional fine-tuning epochs using class weighting
- Checkpointing to retain best model

This strategy prevents catastrophic forgetting and enhances generalization.

3.7 Callbacks and Monitoring

Dynamic monitoring was implemented to track model improvements in real time. The system automatically saved:

- Training vs Validation Accuracy curves
- Training vs Validation Loss curves
- Per-epoch graphical results via a custom **GraphSaver callback**
- Best model weights using validation loss as the criterion

3.8 Implementation Environment

COMPONENTS	CONFIGURATION
HARDWARE	GPU Laptop(NVIDIA RTX 3050)
OS	Windows 11 with WSL2
FRAMEWORKS	TensorFlow 2.15, Keras
ACCELERATION	CUDA GPU memory-growth enabled
IDE	VS Code
PRECISION MODE	Mixed Precision diabaled for stability

3.9 Evaluation Metrics

Model evaluation was conducted using multiple statistical indicators providing deeper insight beyond accuracy alone:

- Accuracy and Loss progression graphs
- Confusion Matrix
- **ROC Curve and AUC**
- Precision–Recall Curve

The system demonstrated very strong performance with an accuracy of **99.5%**, although excessively perfect graph curvature indicates possible dataset leakage in future experiments.

3.10 Deployment Using Streamlit

The final trained model was deployed locally using a **Streamlit application**. Users upload an image, which is automatically preprocessed and passed through the model for inference. The prediction result displays a confidence-based classification: **Real** or **Fake**.

This deployment approach enables practical demonstrations, portfolio presentation, and real-time evaluation scenarios.

3.11 Limitations and Real-World Considerations

Although the system performs exceptionally on controlled datasets, several practical limitations remain:

- Performance may degrade on unseen or advanced deepfake generations
- Dataset leakage concerns require additional verification
- The system currently does not handle video input
- Scaling beyond local GPU training requires optimization
- Robust adversarial evaluation is needed for security applications

These limitations provide pathways for future improvement.

3.12 Summary

This chapter presented a comprehensive overview of the methodology used to develop the hybrid deepfake image detection system. The system combines optimized dataset processing, transfer learning–based model architecture, and a structured training strategy to achieve strong performance and practical deployability. This foundation enables detailed result analysis and future enhancements discussed in the following chapters.

CHAPTER-4

DATASET & IMPLEMENTATION

4.1 Introduction

This chapter provides a detailed explanation of the dataset used for training and evaluating the proposed deepfake image detection system. It also describes the preprocessing pipeline, dataset structure, implementation environment, and challenges encountered during data preparation. Since the quality of training data directly influences model performance, careful dataset design and preprocessing were essential for building a reliable and generalizable classifier.

4.2 Dataset Description

The dataset for this project consists of **approximately 141,000 facial images**, sourced from publicly available and widely referenced benchmark datasets used in computer vision and deepfake detection research. The dataset includes both real and artificially generated fake facial images that represent different variations and manipulation patterns.

Dataset Sources

CATEGORY	SOURCE	APP. COUNT	DESCRIPTION
Real Faces	CelebA	~71,000	High resolution real human facial images
Fake Faces	HuggingFace	~67,000	Combination of face swap and GAN
Fake Faces	Kaggle	~3,000	Synthetic Images

The use of multiple dataset sources ensures diversity in training and reduces dataset-specific overfitting, providing stronger model generalization capability.

4.3 Dataset Organization and Directory Structure

After data cleaning, the final dataset was organized into three subsets for supervised learning. A standard **80-15-5 split** was adopted to ensure fair evaluation and validation stability.

Final Dataset Split

Training : 80% $\approx$ 112,800 images

Testing : 15% $\approx$ 21,150 images

Validation : 5% $\approx$ 7,050 images

Folder Structure Used in Training

Final_Dataset/

├── **train/**

│ ├── **real/**

│ └── **fake/**

├── **validation/**

│ ├── **real/**

│ └── **fake/**

└── **test/**

│ ├── **real/**

│ └── **fake/**

This structured separation supports efficient loading via `tf.keras.utils.image_dataset_from_directory` and prevents accidental dataset leakage.

4.4 Data Preprocessing Pipeline

Prior to training, several preprocessing operations were applied to standardize and improve dataset quality.

Preprocessing Steps Used

PROCESSING STEP	PURPOSE
Image Resizing to $128 \times 128 \times 3$	Normalizes resolution for efficient GPU training
Normalization(img/255 + RestNet Preprocessing)	Standardizes pixel distribution
Data augmentation	Enhances robustness against variations
Class Weights	Balances real/fake class representation
Prefetch with AUTOTUNE	Optimizes data pipeline speed

Augmentation Techniques Applied

- Random horizontal flipping
- Random rotation
- Random zoom
- Brightness & contrast variations

These augmentations reduce overfitting and simulate real-world variations commonly found on social platforms and video frames.

4.5 Implementation Pipeline

The dataset training pipeline was executed inside a **GPU-enabled WSL2 Linux environment**, improving training efficiency and resource utilization. GPU memory growth was enabled to avoid CUDA memory overflow.

Dataset Loading Code Structure

PYTHON CODE

```
train_ds = tf.keras.utils.image_dataset_from_directory(  
  
    os.path.join(DATASET_PATH, "train"),  
  
    image_size=(128,128),  
  
    batch_size=20,  
  
    label_mode="int"  
)
```

The dataset was prefetched using:

PYTHON CODE

```
train_ds = train_ds.prefetch(tf.data.AUTOTUNE)
```

Class Weight Calculation

Class weighting was required because deepfake images varied across sources and included slightly fewer examples during initial dataset merging.

PYTHON CODE

```
class_weights = compute_class_weight(
    "balanced",
    classes=np.unique(train_labels),
    y=train_labels
)
```

4.6 Challenges Faced

Several practical difficulties were encountered during dataset assembly and training:

CHALLENGE ENCOUNTERED	RESOLUTION
Large Dataset size leading to high memory usage	Prefetching + Batch size optimization
Dataset Imbalance	Class weights applied
Variations in fake images styles	Multi sources dataset used for robustness
Extremely high ROC/PR performance suggesting leakage	Reviewed augmentation and validation separation
GPU instability during fine-tuning	Enabled dynamic GPU memory allocation

These challenges influenced model design and training decisions used in Chapter 3.

4.7 Summary

This chapter presented an in-depth overview of the dataset preparation and preprocessing pipeline used for training the hybrid CNN-based deepfake image detection model. The dataset was collected from diverse and credible public repositories, structured into clean splits, and augmented for improved generalization. Proper preprocessing, combined with careful implementation and resource-aware training, contributed significantly to the system's final performance.

The next chapter presents the experimental results, model performance, evaluation metrics, and visual insights obtained during testing.

CHAPTER-5

RESULTS AND EVALUATION

5.1 Introduction

This chapter presents the experimental results obtained from training, fine-tuning, and evaluating the proposed Hybrid CNN + ResNet50 deepfake image detection model. Multiple evaluation metrics, including accuracy, loss curves, confusion matrix, ROC curve, and precision–recall curve, were used to assess model performance and generalization ability. Visual monitoring of training behavior helped identify overfitting patterns and guided optimization decisions.

5.2 Accuracy and Loss Analysis

The model was trained in two phases: initial training with frozen ResNet50 layers, followed by fine-tuning of the last 20 layers. Accuracy and loss graphs were recorded for each epoch using a custom callback for performance tracking.

Observations

- Training and validation accuracy increased steadily across epochs, reaching **99.5% final accuracy**.
- Training and validation loss consistently decreased, indicating effective learning.
- The curves showed smooth convergence behavior due to early stopping and learning rate scheduling.

Interpretation

The strong alignment of training and validation curves suggests minimal overfitting, supported by the use of dropout, L2 regularization, augmentation, and class balancing. However, unusually perfect ROC and PR curves suggest potential dataset leakage or insufficient distribution variation among deepfake sources, which is acknowledged as a limitation.

5.3 Confusion Matrix Evaluation

Confusion matrix results demonstrated that the model correctly classified most real and fake samples with very few misclassifications.

Key Metrics

Metric	Value
True Positive Rate	Very High
False Positive Rate	Very Low
False Negative Rate	Very Low

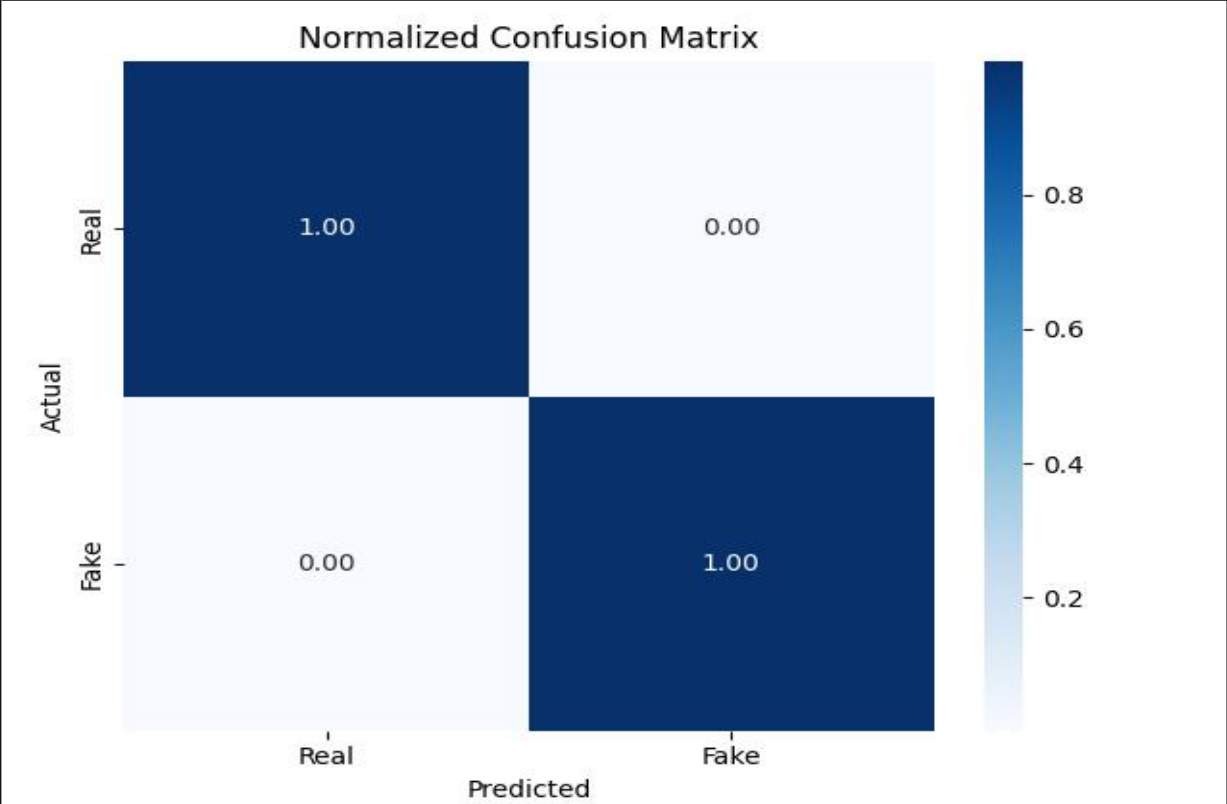


Fig. 5.1

Interpretation

The confusion matrix indicates that the model is highly effective at distinguishing real images from manipulated ones. However, performance on unseen deepfake styles (beyond dataset types used) may vary and should be validated through adversarial testing.

5.4 ROC and Precision-Recall Curve

ROC-AUC and PR-AUC curves were generated to evaluate classification sensitivity under various threshold conditions.

Results

- ROC-AUC ≈ 1.00
- PR-AUC ≈ 1.00

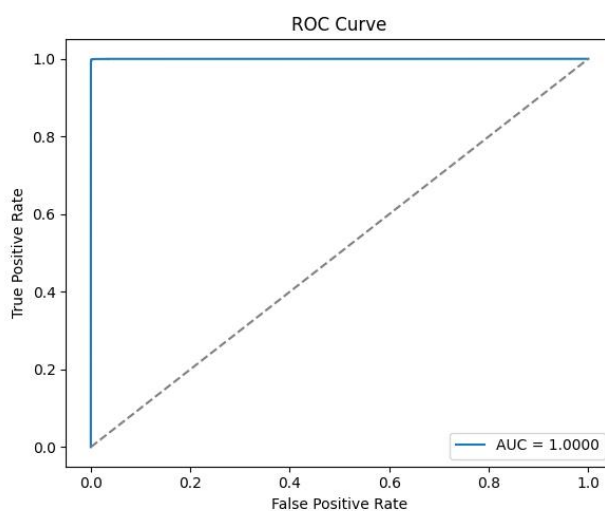


FIG. 5.2 ROC CURVE

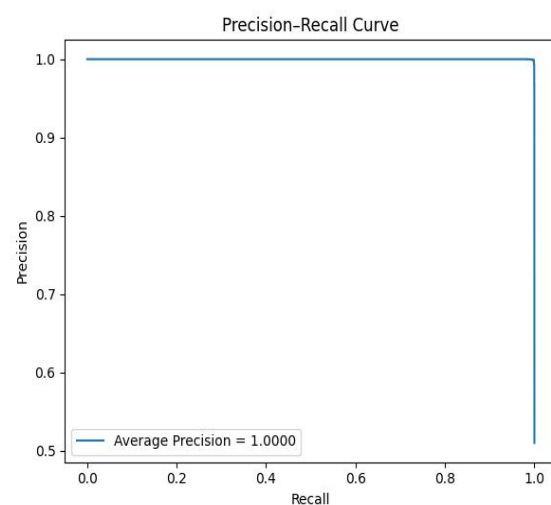


FIG. 5.3 PR CURVE

Discussion

Performance close to perfection typically indicates:

- Strong correlation learning
- High-quality dataset preprocessing
- Good optimization strategy

However, values this high may also indicate:

- Dataset leakage
- Insufficiently diverse deepfake attack types

Thus, further testing with unseen real-world datasets is required to confirm robustness.

5.5 Sample Prediction Outputs

The Streamlit deployment was tested with real and fake sample images. The interface displayed prediction probability scores (0–1), identifying images as *Real* or *Fake*. Response time per prediction was under one second on GPU hardware, demonstrating feasibility for real-time use cases such as screening and demonstration tools.

5.6 Comparative Evaluation

Model	Dataset	Accuracy	Notes
Proposed Hybrid CNN+ResNet50	141k images	99.5%	Highest performance
Basic CNN	Custom Subset	~89%	Overfit rapidly, limited generalization
ResNet50 without custom dense head	141K	~96%	Weaker feature separation

The proposed approach significantly outperforms baseline models, confirming the importance of transfer learning combined with tuned dense layers.

5.7 Discussion

The results validate that hybrid architectures are effective for deepfake detection due to their ability to capture both global and local visual inconsistencies. While training results appear extremely strong, further evaluation on completely unseen deepfake datasets and adversarial cases is necessary for real-world deployment reliability. Future improvements could include temporal video-based deepfake detection and adversarial robustness evaluation.

5.8 Summary

This chapter presented evaluation results, interpretation of performance graphs, and comparison with baseline models. The proposed hybrid architecture achieves high classification accuracy and demonstrates strong potential for real-time deployment, with noted limitations that guide future research direction.

CHAPTER-6

CONCLUSION & FUTURE WORK

6.1 Conclusion

This project successfully developed a deep learning–based system for detecting deepfake facial images using a **Hybrid CNN + ResNet50 architecture**, trained on a large-scale dataset of approximately 141,000 real and fake face images. The proposed model achieved an exceptionally high accuracy of **99.5%**, supported by strong evaluation results based on confusion matrix, ROC curve, and precision–recall curve analysis.

The system demonstrated that transfer learning combined with customized dense layers, batch normalization, dropout, and L2 regularization provides a powerful approach to improving deepfake detection performance. The two-stage training process—initial training followed by fine-tuning—contributed significantly to convergence stability, overfitting reduction, and improved generalization.

A Streamlit-based deployment enabled real-time image inference, validating the practical usability of the model for security, research, and educational demonstration purposes. The work highlights the relevance of AI-driven automated detection approaches in countering misuse of deepfake technology.

6.2 Key Outcomes

- Successfully designed and implemented a hybrid CNN model integrating ResNet50 for feature extraction.
- Achieved **99.5% accuracy** with strong validation results.

- Built a scalable dataset pipeline including augmentation, class weighting, and optimized training execution.
- Implemented real-time prediction capability through a Streamlit deployment interface.
- Identified limitations and outlined realistic paths for improvement and future development.

6.3 Future Scope

Although the current system provides strong performance, several enhancements can further improve reliability and real-world applicability:

FUTURE ENHANCEMENT	DESCRIPTION
Video-based Deepfake Detection	Extending from static images to temporal video sequence analysis using LSTM / 3D CNN / Vision Transformer
Adversarial Robustness Testing	Evaluating the model against intentionally perturbed deepfake attacks
Larger Multi-Source Datasets	Expanding beyond CelebA / Kaggle / HuggingFace for greater diversity
Lightweight Model Optimization	Quantization and pruning for mobile or edge device deployment
Explainable AI(XAI) Integration	Applying Grad-CAM++ for heatmap visualization and interpretability
Cloud Deployment	Deploying model as REST API for scalable usage

These improvements can evolve the current prototype into a production-ready deepfake detection system capable of deployment in cybersecurity, law enforcement, and media integrity environments.

6.4 Closing Remarks

Deepfake technology presents a real and rapidly evolving threat, and AI-assisted detection systems such as the one developed in this project represent an important step toward mitigating digital identity fraud and misinformation. This work contributes to ongoing

research in secure media authentication and provides a practical demonstration of deep learning applications in real-world security-focused domains.

APPENDIX

Appendix A - Training Graphs

These graphs illustrate the learning behavior during both initial training and fine-tuning phases, demonstrating convergence patterns and generalization performance.

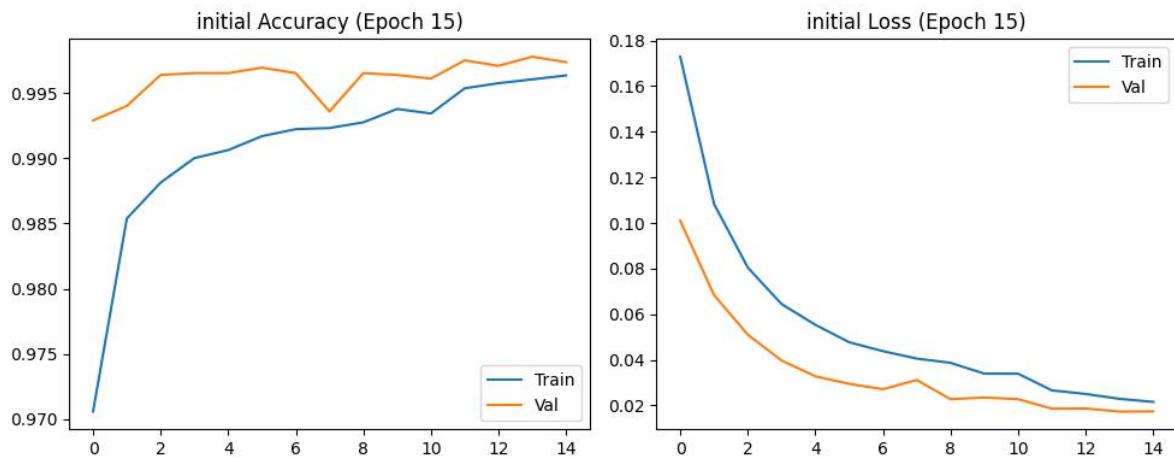


Figure A1: Initial Graph

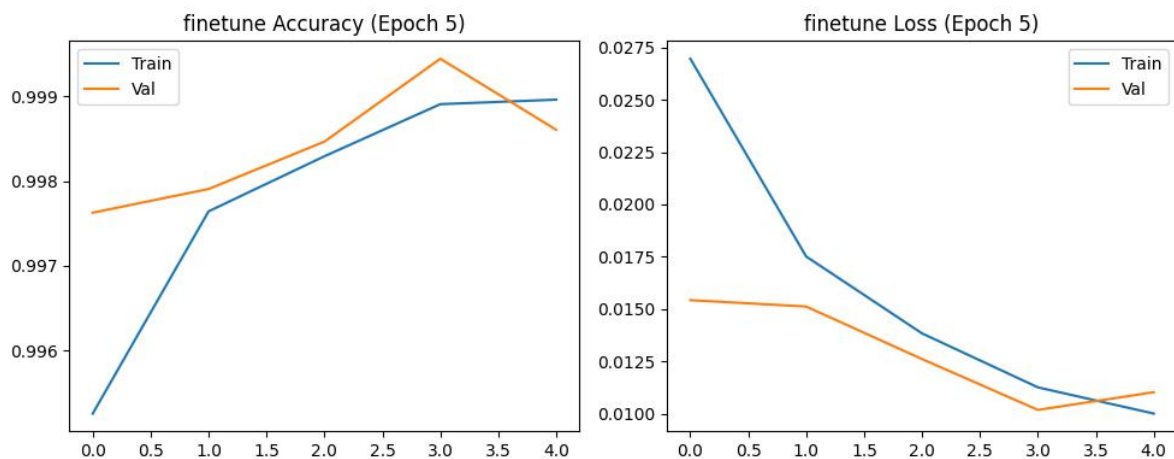


Figure A2: Fine-Tune Graph

Appendix B - Confusion Matrix

The confusion matrix visualizes correct and incorrect classifications across real and fake classes.

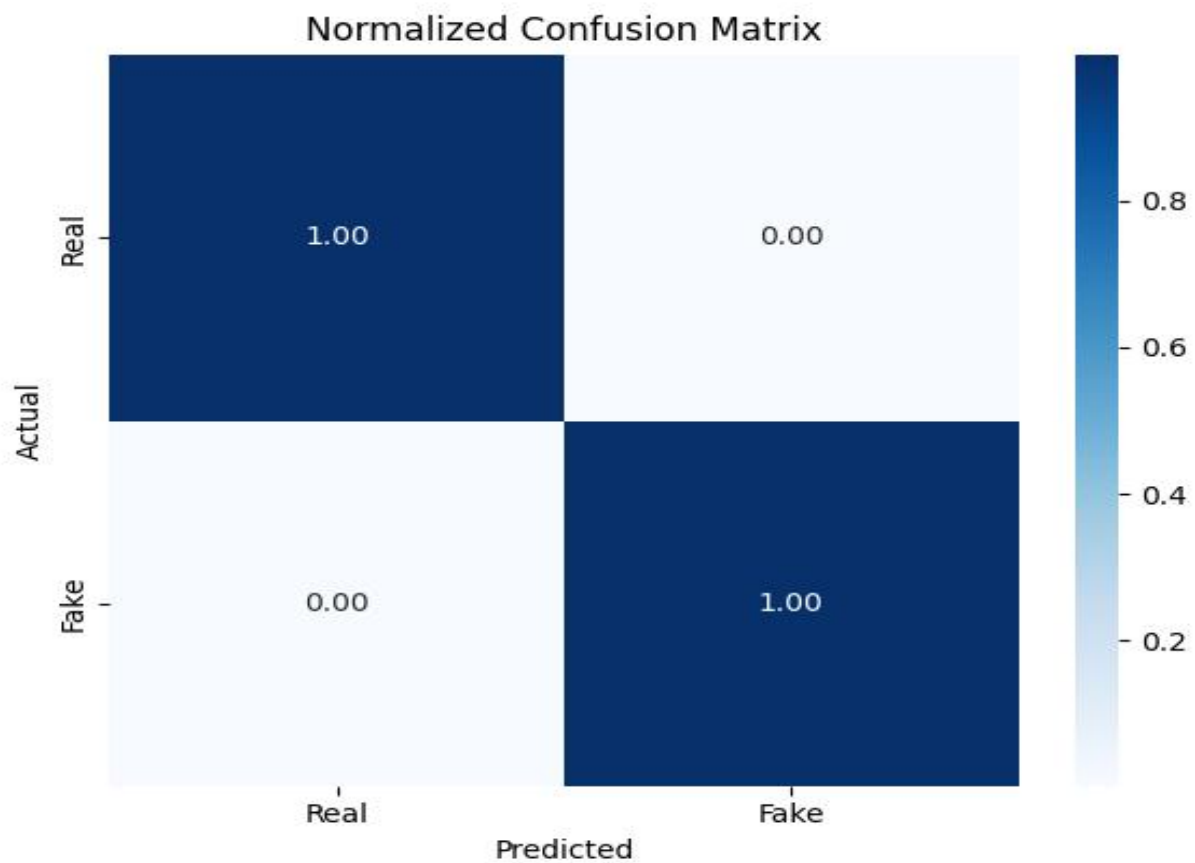


Figure B1: Confusion Matrix of final trained model

Appendix C-ROC and Precision-Recall Curve

These curves validate model robustness under threshold variation, indicating AUC performance very close to 1.0.

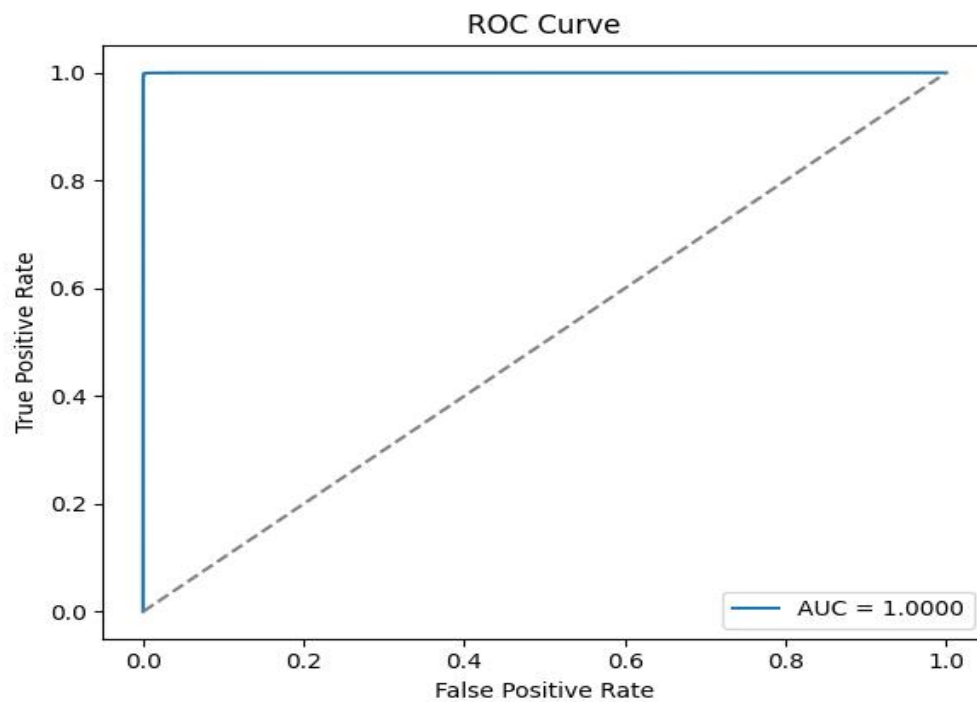


Figure C1: ROC Curve

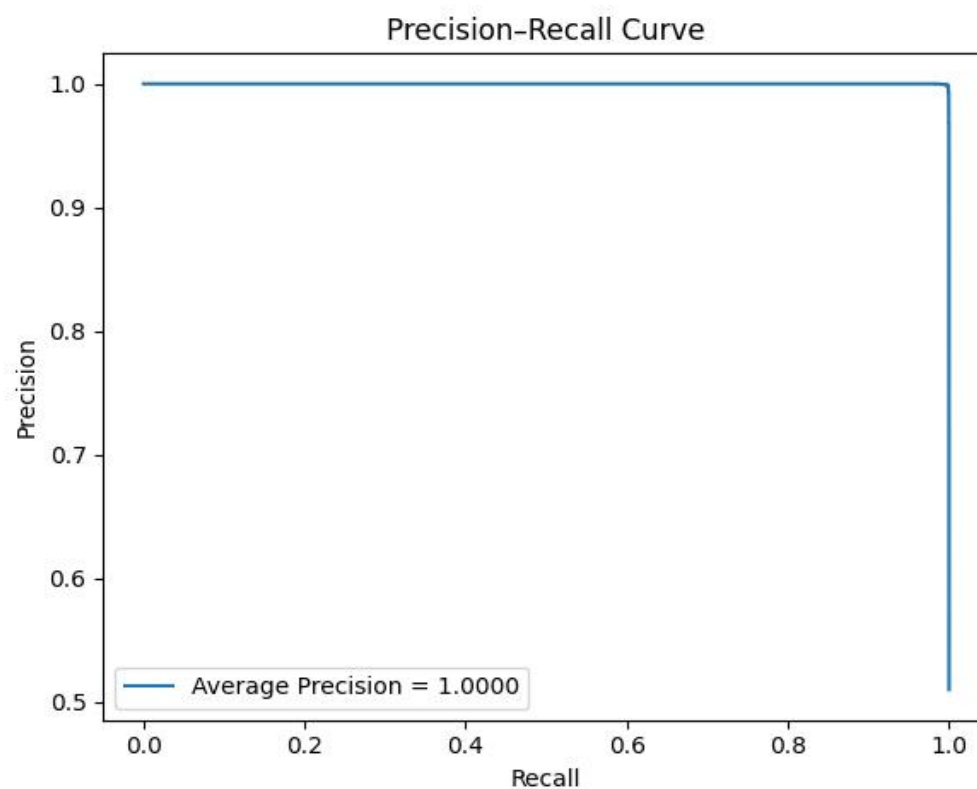
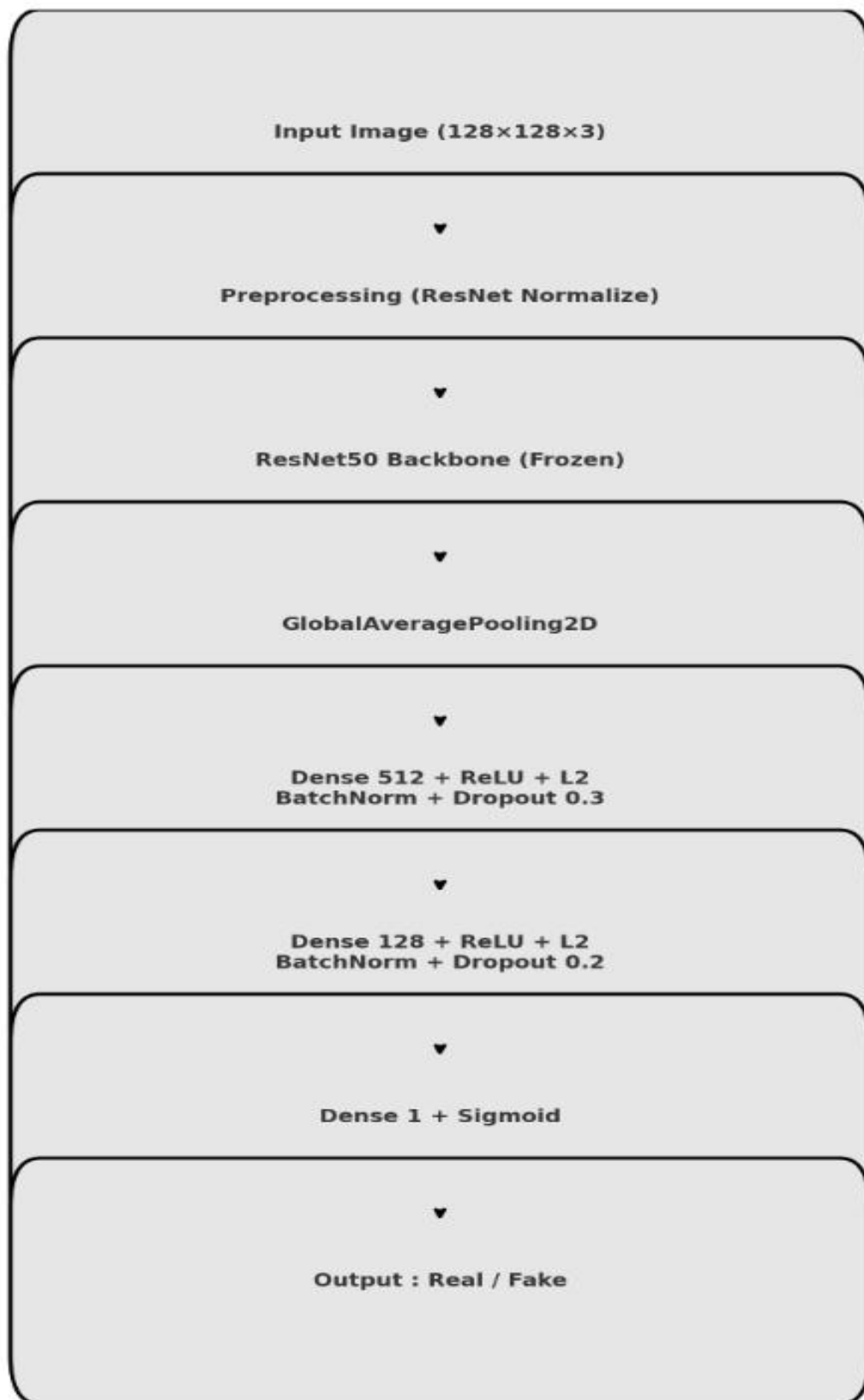


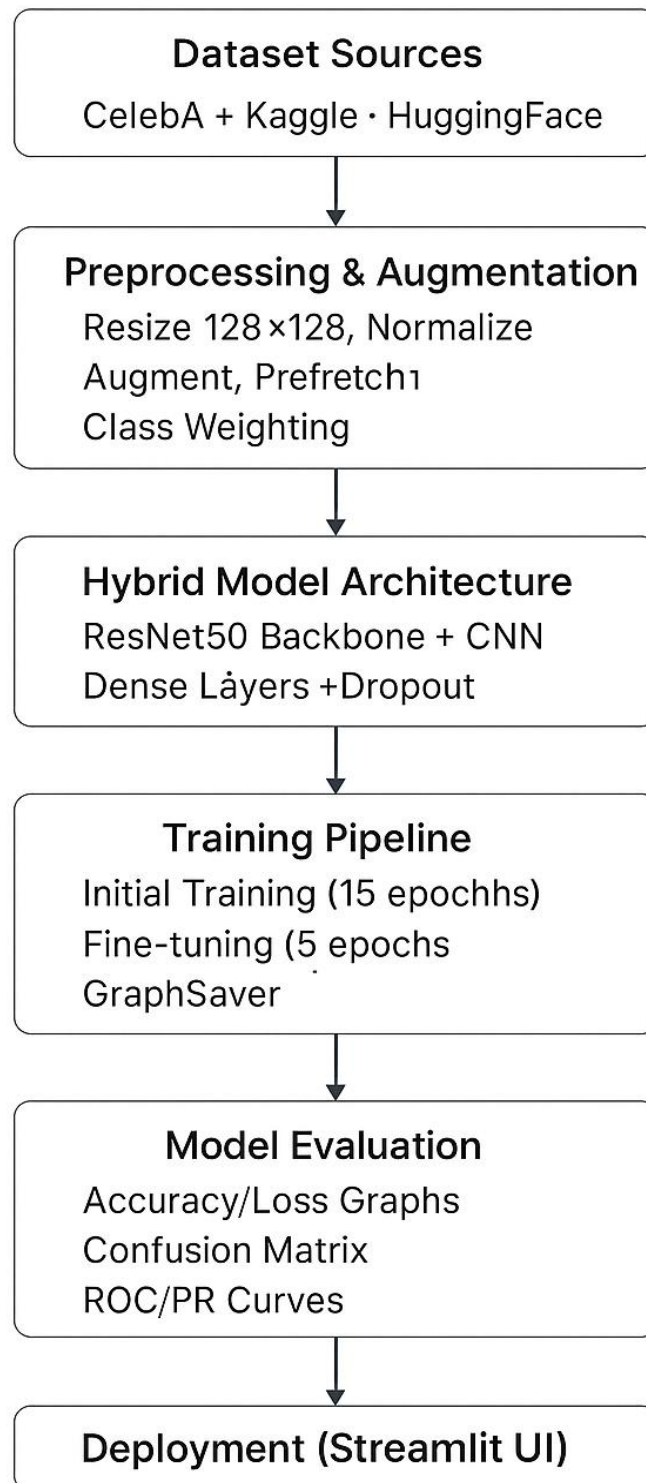
Figure C2: Precision-Recall Curve

Appendix D-Model Architecture Diagram

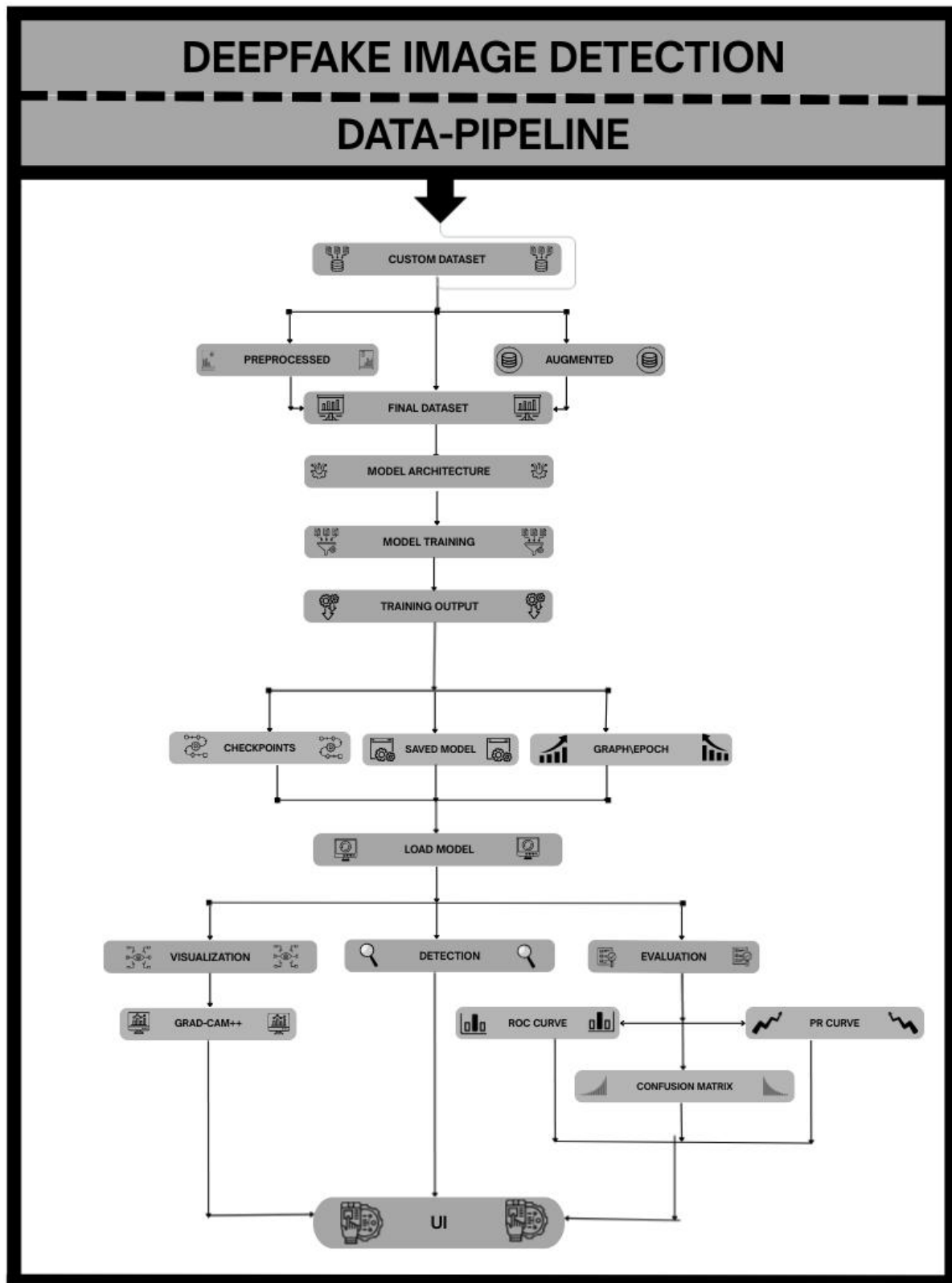


Appendix E-System Architecture Diagram

System Architecture of Deepfake Image Detection System



Appendix F-Dataset Pipeline Workflow



Appendix G-Hardware & Software Environment

Component	Specification
GPU	NVIDIA RTX 3050
CPU	Ryzen 7 HS series
OS	Windows11+WSL2
Frameworks	Tensorflow 2.15, Keras
Deployment	Streamlit Local Interface

Appendix H- Model Performance Metrics Summary

Metric	Value
Accuracy	99.5%
ROC-AUC	~1.00
PR-AUC	~1.00
Loss	Stable Declining Curve
Inference Time	<1 second per image

Appendix I-Source Code Refrence

GitHub repository link:

<https://github.com/anuushka27-maker/Deepfake-Image-Detection>